CARTOGRAPHER

Open-Source Spatial Modeling System

Engineering Architecture, Implementation, Verification, and Repository Extraction Specification

Document	Master Engineering Specification
Scope	General-purpose modeling, DCC, CAD, architecture, animation, rendering, and interchange
Primary implementation	C++23 native core; Vulkan-first renderer; optional Python/C# adapters where useful
Source basis	Inspected Hidden Canopy VANTA Engine, Open World Model Harness, ForgeOfFate, and related repository structure
Open-source posture	Standalone-first. No private VANTA repository required to build or run Cartographer.
Revision	Initial architecture package - September 2026

This document deliberately separates generic infrastructure that can be extracted or independently reimplemented from private VANTA-specific systems. The target is a real modeling application with production-oriented architecture, not a Blender/Maya/AutoCAD feature checklist.

Executive Summary

Cartographer should be built as a standalone open-source modeling environment for anything that exists in space: characters, products, mechanical parts, buildings, terrain, vehicles, props, environments, and technical assemblies. The product is not an AI application. Its primary value is a coherent authoring model that combines artistic polygon workflows, non-destructive DCC evaluation, precision CAD/architectural semantics, animation, and reliable interchange in one native desktop system.

Repository inspection shows that Hidden Canopy already owns a substantial generic substrate inside VANTA: a C++23 world and component model, validated transforms and hierarchy, mesh assets and glTF import, backend-neutral RHI and frame graph, Vulkan rendering, offscreen viewport ownership, project serialization/migrations, asset references, a platform-neutral Studio session, selection, inspector editing, and a reversible command bus. Those pieces dramatically reduce the amount of infrastructure Cartographer must invent, but they should not be exposed by simply open-sourcing VANTA. The correct path is a controlled extraction or clean reimplementation into public generic libraries with independent namespaces, provenance, tests, and no ML/generation/trace dependency.

The largest missing technical body is the actual authoring geometry stack: editable topology, half-edge adjacency, modeling operators, modifiers, dependency evaluation, UV authoring, sculpt/retopo, animation/rigging, CAD sketching and constraints, B-Rep/solid modeling, and domain layers such as architecture. This specification organizes that work into sixty engineering subsystems, each with a design sheet and a verification/delivery sheet. The resulting architecture is deliberately staged so release 0.1 can be a credible mesh-modeling editor before the project attempts deep CAD or BIM breadth.

Core architectural stance

- Authoritative authoring data is distinct from evaluated geometry, renderer resources, caches, and UI state.
- EditableMesh/BRep/Curve data compiles or tessellates into a compact render mesh; the renderer never becomes the geometry database.
- Every user-facing mutation passes through transactions/commands so undo, validation, and diagnostics remain coherent.
- Asynchronous evaluation publishes only revision-current results.
- Project save is atomic, versioned, and migration-tested; crash recovery is separate from named project files.
- Open-source builds require no private repository, credentials, cloud service, or AI runtime.

Inspected Repository Evidence and Reuse Map

Private source path / target	Observed capability	Cartographer use
VANTA: vanta_world; include/vanta/world/world.hpp	Generational entities, component stores, hierarchy, typed mutations, snapshots, replay	Concept/code candidate for generic scene foundation after provenance review; authoring precision upgraded to double.
VANTA: include/vanta/world/components.hpp	TRS transforms, camera, light, rigid body, collider, parent	Baseline semantic reference; expand transform/pivot/negative-scale policy.
VANTA: vanta_assets; mesh_asset.hpp; gltf_import.cpp	Validated MeshAsset, normals, UV0, tangents, bounds, glTF import	Keep as compiled/runtime mesh concept; add independent EditableMesh topology layer.
VANTA: vanta_render; viewport.hpp; render_scene.cpp; renderer.cpp	Offscreen viewport, RenderScene, renderer, resource handles	Strong candidate for public render foundation after private-specific cleanup.
VANTA: vanta_gpu_graph + vanta_rhi	Frame graph and backend-neutral GPU interfaces	Use for viewport rendering and compute-like editor passes; keep authoring model independent.
VANTA: vanta_project; project_document.hpp	Versioned project model, asset refs, atomic save, migrations	Direct architectural precedent for Cartographer project format.
VANTA: vanta_studio_session; command_bus.hpp; inspector_model.hpp	Platform-neutral Studio model, selection, undo/redo commands, inspector facade	Direct precedent for editor command architecture and thin shell.
Open-World-Model-Harness integrations/unreal + integrations/unity	Real Unreal/Unity adapters and world integration experience	Downstream interchange/integration patterns only; not a geometry dependency.
ForgeOfFate Unreal project	Real consumer project with Source/Content/Tools	Internal downstream validation corpus for export compatibility.

The public codebase should not retain VANTA module names or depend on private build targets. The extraction plan in this document treats inspected code as implementation evidence and a source of proven patterns, then re-homes only generic functionality under Cartographer/public-library ownership.

Target Public Repository Family

A clean public ecosystem is preferable to one giant repository if the libraries remain independently useful. The recommended family is: Cartographer for the application; Canopy Geometry for editable mesh/curves/CAD geometry; Canopy Scene for object identity, hierarchy, document-neutral scene state, transactions, and serialization primitives; Canopy Render for RHI/frame graph/viewport/render scene; and Canopy Exchange for format adapters. The final names can change, but the separation keeps VANTA proprietary while making the public infrastructure genuinely reusable.

Repository	Owns	Must not own
Cartographer	Desktop shell, workspaces, tools, editor UX, project orchestration	Private VANTA runtime, defense/governance systems
Canopy Geometry	Editable mesh, topology, curves, modifiers, CAD kernel interfaces	Application panes, ML, project-specific UI
Canopy Scene	Scene IDs, hierarchy, transforms, transaction primitives, generic serialization	Rendering backend, DCC-specific UI
Canopy Render	RHI, frame graph, viewport, renderer, GPU resources	Authoritative editable geometry
Canopy Exchange	Import/export adapters and reports	Application model ownership

Table of Contents

01. Product Definition and Scope - Design Specification / Verification and Delivery Specification
02. Open-Source Boundary and IP Separation - Design Specification / Verification and Delivery Specification
03. Build and Module Architecture - Design Specification / Verification and Delivery Specification
04. Application Shell and Workspaces - Design Specification / Verification and Delivery Specification
05. Project Document Model - Design Specification / Verification and Delivery Specification
06. Scene Graph and Object Hierarchy - Design Specification / Verification and Delivery Specification
07. Entity and Component Storage - Design Specification / Verification and Delivery Specification
08. Transform and Hierarchy Evaluation - Design Specification / Verification and Delivery Specification
09. Command Bus, Transactions, Undo and Redo - Design Specification / Verification and Delivery Specification
10. Selection, Modes, and Tool Context - Design Specification / Verification and Delivery Specification
11. Editable Mesh Core - Design Specification / Verification and Delivery Specification
12. Half-Edge Topology - Design Specification / Verification and Delivery Specification
13. Topology Operators - Design Specification / Verification and Delivery Specification
14. Normals, Tangents, Bounds, and Derived Mesh Data - Design Specification / Verification and Delivery Specification
15. Modifier Stack - Design Specification / Verification and Delivery Specification
16. Dependency Graph and Evaluation - Design Specification / Verification and Delivery Specification
17. Subdivision Surfaces - Design Specification / Verification and Delivery Specification
18. Curves and Splines - Design Specification / Verification and Delivery Specification
19. NURBS Curves and Surfaces - Design Specification / Verification and Delivery Specification
20. Sculpting - Design Specification / Verification and Delivery Specification
21. Retopology and Surface Snapping - Design Specification / Verification and Delivery Specification
22. UV Editing and Mapping - Design Specification / Verification and Delivery Specification
23. Materials and Shading Model - Design Specification / Verification and Delivery Specification
24. Node Material Graph - Design Specification / Verification and Delivery Specification
25. Texture and Image Pipeline - Design Specification / Verification and Delivery Specification
26. Renderer and RHI - Design Specification / Verification and Delivery Specification
27. Viewport and Editor Overlays - Design Specification / Verification and Delivery Specification
28. Lighting, Cameras, and Display Modes - Design Specification / Verification and Delivery Specification
29. Asset Database and Libraries - Design Specification / Verification and Delivery Specification
30. Import Pipeline - Design Specification / Verification and Delivery Specification
31. Export Pipeline - Design Specification / Verification and Delivery Specification
32. CAD Sketching - Design Specification / Verification and Delivery Specification
33. Geometric Constraint Solver - Design Specification / Verification and Delivery Specification
34. B-Rep Geometry Kernel - Design Specification / Verification and Delivery Specification
35. Solid Modeling and Booleans - Design Specification / Verification and Delivery Specification
36. Parametric History and Feature Tree - Design Specification / Verification and Delivery Specification
37. Assemblies and Instancing - Design Specification / Verification and Delivery Specification
38. Architecture and BIM Objects - Design Specification / Verification and Delivery Specification
39. Site, Terrain, and Large Spatial Models - Design Specification / Verification and Delivery Specification
40. Measurements, Units, Snapping, and Precision Input - Design Specification / Verification and Delivery Specification
41. Animation Timeline and Curves - Design Specification / Verification and Delivery Specification
42. Rigging and Skinning - Design Specification / Verification and Delivery Specification
43. Deformers and Blend Shapes - Design Specification / Verification and Delivery Specification

44. Physics and Simulation - Design Specification / Verification and Delivery Specification
45. Project Persistence, Migration, and Atomic Save - Design Specification / Verification and Delivery Specification
46. Autosave and Crash Recovery - Design Specification / Verification and Delivery Specification
47. Plugin System - Design Specification / Verification and Delivery Specification
48. Python Scripting API - Design Specification / Verification and Delivery Specification
49. C++ SDK and Embedding - Design Specification / Verification and Delivery Specification
50. Headless CLI and Batch Processing - Design Specification / Verification and Delivery Specification
51. Performance and Memory Architecture - Design Specification / Verification and Delivery Specification
52. Threading and Job System - Design Specification / Verification and Delivery Specification
53. Large Scene and Streaming Strategy - Design Specification / Verification and Delivery Specification
54. Testing, Fuzzing, and Verification - Design Specification / Verification and Delivery Specification
55. CI, Packaging, and Releases - Design Specification / Verification and Delivery Specification
56. Licensing and Community Governance - Design Specification / Verification and Delivery Specification
57. VANTA Extraction Plan - Design Specification / Verification and Delivery Specification
58. Open World Harness and Forge Integration - Design Specification / Verification and Delivery Specification
59. Roadmap from 0.1 to 1.0 - Design Specification / Verification and Delivery Specification
60. Risk Register and Final Acceptance - Design Specification / Verification and Delivery Specification

Architecture Overview

The architecture is intentionally layered so a modeling operation can be traced from user input to durable project state and then to derived viewport output. The central rule is that rendered geometry is an output of the document/evaluation graph, not a second source of truth.

Layer	Representative modules	Direction of dependency
Application	Workspaces, panels, actions, tools, project controller	Down into editor/services only
Editor	Selection, command bus, transactions, inspectors, modes	Into document/geometry APIs
Authoring data	Scene, EditableMesh, Curve, Sketch, BRep, Animation, Materials	Owns durable truth
Evaluation	Dependency graph, modifiers, constraints, tessellation, deformation	Reads truth; emits revision-tagged derived results
Runtime/render data	Compiled MeshAsset, RenderScene, GPU resources, BVH	Derived and disposable
Platform/backends	Vulkan, windowing, file system, plugin loader	No authority over document model

```
User action
   -> Tool/Operator
      -> EditorCommand / Transaction
         -> Authoring Document
            -> Dependency Graph
               -> Evaluated Geometry
                  -> Compiled Mesh / RenderScene
                     -> Viewport / Export / Simulation
```

01. Product Definition and Scope - Design Specification

Define Cartographer as a standalone open-source spatial authoring system spanning polygonal DCC, precision CAD, architectural modeling, animation, and technical output without making AI a product dependency.

Source basis: New product. Reuse generic VANTA infrastructure only after extraction into neutral libraries.

Design intent

This subsystem exists to make product definition and scope a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- One coherent scene/document model across workspaces. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- non-destructive authoring first. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- local-first desktop operation. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- AI optional and external. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- Maya/Blender-class modeling behavior is the reference for interaction depth, not binary compatibility.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
ApplicationShell	Own or mediate the ApplicationShell concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
WorkspaceRegistry	Own or mediate the WorkspaceRegistry concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DocumentController	Own or mediate the DocumentController concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CapabilityRegistry	Own or mediate the CapabilityRegistry concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ProjectMetadata	Own or mediate the ProjectMetadata concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- A project opens without network access. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- every edit is reversible or explicitly marked destructive. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- workspace switching never changes model meaning. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- engine/runtime data is separated from authoring data.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

01. Product Definition and Scope - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Feature sprawl creates incompatible object types. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- CAD and mesh paths diverge. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- UI directly mutates model state. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- hidden VANTA-specific dependencies leak into public code.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Open/save empty and complex projects	Automate the Open/save empty and complex projects case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
workspace switching	Automate the workspace switching case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
offline launch	Automate the offline launch case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
undo across workspace boundaries	Automate the undo across workspace boundaries case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
license/dependency audit.	Automate the license/dependency audit. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Product charter. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- scope matrix. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

- exclusion list. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- architecture decision records. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- v0.1 capability statement.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Product Definition and Scope is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

02. Open-Source Boundary and IP Separation - Design Specification

Establish what can be extracted or reimplemented from private repositories without exposing VANTA-specific ML, generation, trace, defense, or governance IP.

Source basis: VANTA CMake separation already isolates vanta_world, vanta_assets, vanta_render, vanta_project, and vanta_studio_session from ML/generation consumers.

Design intent

This subsystem exists to make open-source boundary and ip separation a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Copy only generic mechanisms after provenance review. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- preserve VANTA behavior through adapter tests rather than shared private headers. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- public repos use independent namespaces and release histories. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- no private submodule requirement.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
ExtractionManifest	Own or mediate the ExtractionManifest concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ProvenanceRecord	Own or mediate the ProvenanceRecord concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CompatibilityAdapter	Own or mediate the CompatibilityAdapter concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
LicenseManifest	Own or mediate the LicenseManifest concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Public build succeeds with zero access to private repositories. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- each extracted file has origin/provenance. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- private-only types never appear in public headers.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

02. Open-Source Boundary and IP Separation - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Accidental code disclosure. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- static linkage to private targets. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- copied comments reveal sensitive roadmap. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- incompatible licenses enter dependency graph.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Clean-room build on fresh machine	Automate the Clean-room build on fresh machine case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
grep for private target names	Automate the grep for private target names case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
dependency closure test	Automate the dependency closure test case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
license scanner	Automate the license scanner case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
binary symbol inspection.	Automate the binary symbol inspection. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.

- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Extraction checklist. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- source mapping. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- approved/rejected module list. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- public dependency graph. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- provenance appendix.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Open-Source Boundary and IP Separation is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

03. Build and Module Architecture - Design Specification

Create a C++23 modular build that supports desktop app, headless libraries, tests, plugins, and optional platform backends.

Source basis: VANTA CMake already demonstrates small static/interface targets and backend-neutral seams.

Design intent

This subsystem exists to make build and module architecture a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- CMake 3.25+ baseline. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- libraries compile independently. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- UI backend is not required for geometry tests. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- optional Vulkan and platform shells. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- installable exported targets.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
carto_core	Own or mediate the carto_core concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
carto_scene	Own or mediate the carto_scene concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
carto_geometry	Own or mediate the carto_geometry concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
carto_render	Own or mediate the carto_render concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
carto_project	Own or mediate the carto_project concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
carto_editor	Own or mediate the carto_editor concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
carto_cad	Own or mediate the carto_cad concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
carto_animation	Own or mediate the carto_animation concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
carto_io	Own or mediate the carto_io concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
carto_sdk	Own or mediate the carto_sdk concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- No circular target dependencies. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- public headers compile standalone. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- unit tests link the narrowest target. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- feature options fail closed when dependency absent.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

03. Build and Module Architecture - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Monolithic target growth. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- transitive dependency explosion. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

- platform headers leak into core. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- optional dependency silently changes model semantics.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
CMake graph lint	Automate the CMake graph lint case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
header self-containment	Automate the header self-containment case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
minimal-link tests	Automate the minimal-link tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
Windows/Linux CI	Automate the Windows/Linux CI case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
build option matrix.	Automate the build option matrix. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Root CMake. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- package config. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- target map. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- toolchain presets. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- dependency policy.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Build and Module Architecture is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

04. Application Shell and Workspaces - Design Specification

Define the native desktop shell and the Model, Design, Build, Animate, Material, Scene, Simulate, Draft, and Render workspaces.

Source basis: VANTA Studio provides a thin Win32 shell over a platform-neutral StudioSession.

Design intent

This subsystem exists to make application shell and workspaces a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Workspace is a UI composition, not a data silo. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- commands are globally available through actions. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- panes are dockable. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- layouts persist per user, not per model unless explicitly saved.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
WorkspaceHost	Own or mediate the WorkspaceHost concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DockLayout	Own or mediate the DockLayout concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ActionRegistry	Own or mediate the ActionRegistry concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
PanelFactory	Own or mediate the PanelFactory concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DocumentView	Own or mediate the DocumentView concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Changing workspaces preserves selection and document revision. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- panels do not own authoritative state. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- layout corruption cannot corrupt project files.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

04. Application Shell and Workspaces - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Workspace plugin crashes shell. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- panel writes model directly. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- saved layout references missing plugin. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- modal tools trap input.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Open same project in each workspace	Automate the Open same project in each workspace case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
restore layout with missing panel	Automate the restore layout with missing panel case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
keyboard-only navigation	Automate the keyboard-only navigation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
multi-monitor DPI tests.	Automate the multi-monitor DPI tests. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Shell skeleton. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- workspace registry. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- pane contracts. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- layout persistence. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- action routing.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Application Shell and Workspaces is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

05. Project Document Model - Design Specification

Specify the durable project representation, versioning, editor metadata, external assets, units, and feature capability declarations.

Source basis: VANTA ProjectDocument v8 already separates durable world state, editor metadata, asset references, and migration versions.

Design intent

This subsystem exists to make project document model a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Cartographer format owns authoring state, not render cache. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- schema version increments on semantic changes. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- unknown future versions fail safely. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- external asset paths are project-relative by default.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
ProjectDocument	Own or mediate the ProjectDocument concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SchemaVersion	Own or mediate the SchemaVersion concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
AssetReference	Own or mediate the AssetReference concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
FeatureManifest	Own or mediate the FeatureManifest concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
EditorMetadata	Own or mediate the EditorMetadata concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SceneDocument	Own or mediate the SceneDocument concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Serialization round-trips exactly for supported numeric values. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

- save is atomic. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- load never partially mutates an open document. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- migrations are explicit and tested.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

05. Project Document Model - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Truncated save. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- stale cache serialized as truth. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- migration drops geometry. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- absolute path leaks. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- future version opened incorrectly.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Golden-file round trips	Automate the Golden-file round trips case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
power-loss save simulation	Automate the power-loss save simulation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
migration chain tests	Automate the migration chain tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
corrupted JSON/binary chunks	Automate the corrupted JSON/binary chunks case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
path traversal cases.	Automate the path traversal cases. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.

- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- File-format spec. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- migration registry. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- atomic writer. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- fixtures. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- recovery behavior.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Project Document Model is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

06. Scene Graph and Object Hierarchy - Design Specification

Provide a general scene graph for DCC objects, CAD objects, cameras, lights, rigs, collections, and references.

Source basis: VANTA World supports generational entity IDs, component stores, parent links, cycle rejection, and resolved world transforms.

Design intent

This subsystem exists to make scene graph and object hierarchy a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Stable object identity is independent of array index. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- hierarchy is explicit. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- collection membership is not parentage. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- instances reference shared data blocks. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- editor labels are metadata.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Scene	Own or mediate the Scene concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ObjectId	Own or mediate the ObjectId concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SceneObject	Own or mediate the SceneObject concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ParentLink	Own or mediate the ParentLink concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CollectionId	Own or mediate the CollectionId concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DataBlockId	Own or mediate the DataBlockId concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
InstanceRef	Own or mediate the InstanceRef concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- No hierarchy cycles. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- deleting a parent has defined child policy. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- world transforms resolve deterministically. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- object identity survives serialization.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

06. Scene Graph and Object Hierarchy - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Dangling parent. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- reference cycle. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- duplicate UUID. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- stale instance. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- hierarchy edit invalidates animation incorrectly.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Random hierarchy fuzz	Automate the Random hierarchy fuzz case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
deep hierarchy transform test	Automate the deep hierarchy transform test case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
delete/reparent undo	Automate the delete/reparent undo case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
instance serialization	Automate the instance serialization case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
duplicate ID rejection.	Automate the duplicate ID rejection. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Scene graph library. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- hierarchy commands. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- object registry. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- collection system. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- scene validation.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Scene Graph and Object Hierarchy is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

07. Entity and Component Storage - Design Specification

Define high-performance object/component storage for scene attributes while keeping authoring data types explicit.

Source basis: VANTA component stores and entity registry are proven starting points for generic scene state.

Design intent

This subsystem exists to make entity and component storage a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Use generational handles internally. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- authoring data blocks may be heap-owned and referenced. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- component presence is sparse. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- mutation occurs through command-controlled APIs.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
EntityRegistry	Own or mediate the EntityRegistry concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ComponentStore<T>	Own or mediate the ComponentStore<T> concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ObjectHandle	Own or mediate the ObjectHandle concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DataHandle	Own or mediate the DataHandle concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SceneMutation	Own or mediate the SceneMutation concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Stale handles fail. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- registry does not alias destroyed objects. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- iteration order is deterministic where persisted output depends on it.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

07. Entity and Component Storage - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Use-after-destroy. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- handle reuse ambiguity. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- component store drift. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- direct pointer retention across compaction.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Stale-handle tests	Automate the Stale-handle tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
randomized create/destroy	Automate the randomized create/destroy case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
serialization ordering	Automate the serialization ordering case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
component attach/detach stress.	Automate the component attach/detach stress. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Core registry. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- typed stores. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- validation utilities. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- debug diagnostics.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Entity and Component Storage is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial

parsing or approximate export for full semantic compatibility.

08. Transform and Hierarchy Evaluation - Design Specification

Support full TRS, pivots, parent spaces, constraints, object/local/world coordinates, and CAD-friendly numeric precision.

Source basis: VANTA has validated quaternion TRS and parent-resolved world transforms, but currently uses float world transforms.

Design intent

This subsystem exists to make transform and hierarchy evaluation a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Authoring transforms use double precision. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- render extraction may down-convert. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- pivots are distinct from object origin. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- matrix decomposition failure is explicit. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- negative scale policy is defined.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
TransformD	Own or mediate the TransformD concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
TransformSpace	Own or mediate the TransformSpace concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Pivot	Own or mediate the Pivot concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
WorldMatrixCache	Own or mediate the WorldMatrixCache concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
TransformEvaluator	Own or mediate the TransformEvaluator concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Transforms are finite. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- parent evaluation is acyclic. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- world/local conversion is reversible within tolerance. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- render conversion reports precision loss.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

08. Transform and Hierarchy Evaluation - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Singular scale. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- negative determinant flips normals. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- deep hierarchy precision loss. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- shear cannot decompose to TRS.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Property-based compose/decompose	Automate the Property-based compose/decompose case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
10k-deep hierarchy	Automate the 10k-deep hierarchy case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
large-coordinate precision	Automate the large-coordinate precision case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
negative scale rendering.	Automate the negative scale rendering. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Double-precision transform core. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- cache. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- pivot model. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

- gizmo conversion helpers.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Transform and Hierarchy Evaluation is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

09. Command Bus, Transactions, Undo and Redo - Design Specification

Make every authoring operation reversible, grouped, inspectable, and safe across compound edits.

Source basis: VANTA Studio command bus already routes edits through validated World admission and maintains undo/redo stacks.

Design intent

This subsystem exists to make command bus, transactions, undo and redo a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Commands produce transactions. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- drag tools coalesce continuous edits. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- destructive commands capture sufficient inverse data. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- history can be bounded by memory policy. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- file open resets history boundary.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
EditorCommand	Own or mediate the EditorCommand concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Transaction	Own or mediate the Transaction concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CommandBus	Own or mediate the CommandBus concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
HistoryEntry	Own or mediate the HistoryEntry concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CoalescingScope	Own or mediate the CoalescingScope concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Failed command does not enter history. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- undo uses the same validation path as forward edit. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- redo invalidates after divergent edit. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- grouped command is atomic.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

09. Command Bus, Transactions, Undo and Redo - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Partial multi-object edit. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- inverse data missing. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- history retains huge sculpt buffers. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- plugin command throws during undo.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Failure injection	Automate the Failure injection case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
multi-object atomicity	Automate the multi-object atomicity case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
coalesced transform drag	Automate the coalesced transform drag case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
history memory ceiling	Automate the history memory ceiling case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
plugin exception containment.	Automate the plugin exception containment. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.

- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Generic command framework. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- transaction API. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- history panel data. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- memory accounting.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Command Bus, Transactions, Undo and Redo is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

10. Selection, Modes, and Tool Context - Design Specification

Define object, component, UV, pose, sculpt, CAD sketch, and other selection modes with predictable ownership.

Source basis: VANTA has a basic selection model and inspector integration.

Design intent

This subsystem exists to make selection, modes, and tool context a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Selection is editor state, not project truth unless explicitly saved. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- component selections reference topology generation. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- mode changes validate stale references. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- active element is distinct from selection set.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
SelectionModel	Own or mediate the SelectionModel concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
SelectionSet	Own or mediate the SelectionSet concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ActiveElement	Own or mediate the ActiveElement concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ToolContext	Own or mediate the ToolContext concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ModeRegistry	Own or mediate the ModeRegistry concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- A stale topology selection is rejected after destructive topology change. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- selection changes do not dirty project model. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- active object belongs to current scene.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

10. Selection, Modes, and Tool Context - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Stale face index. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- hidden object remains active. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- cross-mode hotkey applies wrong operator. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- selection undo pollutes model history.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Topology edit invalidation	Automate the Topology edit invalidation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
mode-switch tests	Automate the mode-switch tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
multi-select inspector	Automate the multi-select inspector case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
hidden/locked object selection policy.	Automate the hidden/locked object selection policy. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Selection subsystem. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- mode contracts. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- active-element semantics. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- selection diagnostics.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Selection, Modes, and Tool Context is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

11. Editable Mesh Core - Design Specification

Provide a production authoring mesh distinct from the compact render mesh used by the renderer.

Source basis: VANTA MeshAsset is an appropriate compiled runtime representation, not an editable topology structure.

Design intent

This subsystem exists to make editable mesh core a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- EditableMesh is topological truth. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- compile step produces render MeshAsset. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- per-element stable IDs survive non-destructive edits when possible. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- attributes are layered.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
EditableMesh	Own or mediate the EditableMesh concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
VertexId	Own or mediate the VertexId concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
EdgeId	Own or mediate the EdgeId concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
FaceId	Own or mediate the FaceId concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
LoopId	Own or mediate the LoopId concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
AttributeLayer	Own or mediate the AttributeLayer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
MeshCompiler	Own or mediate the MeshCompiler concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Topology has no dangling half-edges. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- face loops are closed. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- attribute cardinality matches domain. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- compiler never mutates source mesh.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

11. Editable Mesh Core - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Non-manifold input. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- zero-area face. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- duplicate edge. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- invalid polygon winding. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

- compile precision overflow.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Mesh invariant validator	Automate the Mesh invariant validator case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
import malformed meshes	Automate the import malformed meshes case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
compile determinism	Automate the compile determinism case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
attribute propagation tests.	Automate the attribute propagation tests. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Editable mesh library. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- validator. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- runtime compiler. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- debug dump format.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Editable Mesh Core is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

12. Half-Edge Topology - Design Specification

Specify the internal adjacency structure needed for fast modeling operators and manifold diagnostics.

Source basis: New geometry code; no equivalent authoring topology exists in VANTA.

Design intent

This subsystem exists to make half-edge topology a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as

a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Half-edge or BMesh-like representation. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- explicit radial cycles for non-manifold support. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- element pools use stable handles. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- boundary edges are first-class.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
HalfEdgeMesh	Own or mediate the HalfEdgeMesh concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
HEVertex	Own or mediate the HEVertex concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
HEEdge	Own or mediate the HEEdge concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
HEFace	Own or mediate the HEFace concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
HELoop	Own or mediate the HELoop concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
TopologyCursor	Own or mediate the TopologyCursor concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Every loop has valid next/prev. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- radial links are symmetric. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- face boundary is traversable. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- deletion invalidates handles by generation.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

12. Half-Edge Topology - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Broken radial ring. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- self-loop. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- duplicate directed edge. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- face with fewer than three unique vertices.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Invariant fuzzing	Automate the Invariant fuzzing case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
Euler operator tests	Automate the Euler operator tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
manifold/non-manifold fixtures	Automate the manifold/non-manifold fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
randomized delete/restore.	Automate the randomized delete/restore. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Topology kernel. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- traversal API. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- invariant checker. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- graph visualizer.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Half-Edge Topology is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

13. Topology Operators - Design Specification

Implement core polygon modeling actions such as extrude, inset, bevel, bridge, loop cut, dissolve, merge, fill, knife, and triangulate.

Source basis: New authoring layer compiled to VANTA-like MeshAsset for rendering.

Design intent

This subsystem exists to make topology operators a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Operators are parameterized commands. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- preview runs on disposable copy or procedural op node. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- attribute propagation rules are explicit. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- operator output includes selection remap.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
MeshOperator	Own or mediate the MeshOperator concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
OperatorContext	Own or mediate the OperatorContext concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SelectionRemap	Own or mediate the SelectionRemap concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
AttributePropagation	Own or mediate the AttributePropagation concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
OperatorResult	Own or mediate the OperatorResult concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Operator failure leaves mesh unchanged. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- produced topology validates. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- material/UV/normal attributes propagate deterministically.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

13. Topology Operators - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Bevel self-intersection. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- inset collapses. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- bridge mismatched loops. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- knife creates zero-length edge. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- dissolve breaks hole topology.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Golden meshes	Automate the Golden meshes case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
randomized parameters	Automate the randomized parameters case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
undo/redo	Automate the undo/redo case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
attribute preservation	Automate the attribute preservation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
selection remap.	Automate the selection remap. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Operator library. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- preview protocol. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- parameter schemas. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- acceptance gallery.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Topology Operators is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

14. Normals, Tangents, Bounds, and Derived Mesh Data - Design Specification

Centralize derived geometry generation so editor and renderer agree on shading and bounds.

Source basis: VANTA MeshAsset validates normals, tangents, AABB, sphere, and deterministic generation.

Design intent

This subsystem exists to make normals, tangents, bounds, and derived mesh data a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Editable custom normals may override generated normals. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- tangent generation keyed by UV layer. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- derived caches use revision stamps. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- invalidation is local when possible.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
NormalLayer	Own or mediate the NormalLayer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
TangentCache	Own or mediate the TangentCache concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
BoundsCache	Own or mediate the BoundsCache concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DerivedMeshRevision	Own or mediate the DerivedMeshRevision concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Normals are finite. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- tangents orthogonal within tolerance. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- bounds enclose all positions. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

- cache revision matches source geometry/attribute revisions.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

14. Normals, Tangents, Bounds, and Derived Mesh Data - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- NaN from degenerate face. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- stale tangent after UV edit. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- incorrect bounds after modifier. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- mirrored UV handedness error.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Reference vectors	Automate the Reference vectors case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
degenerate cases	Automate the degenerate cases case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
UV mirror	Automate the UV mirror case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
cache invalidation	Automate the cache invalidation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
compiled mesh comparison against VANTA asset rules.	Automate the compiled mesh comparison against VANTA asset rules. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Derived-data service. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- revision keys. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- debug validation. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- compiler bridge.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Normals, Tangents, Bounds, and Derived Mesh Data is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

15. Modifier Stack - Design Specification

Provide non-destructive object-level geometry operations with stable ordering, enable/disable controls, and viewport/render evaluation levels.

Source basis: VANTA has generation and materialization seams but no DCC modifier stack; Cartographer must implement independently.

Design intent

This subsystem exists to make modifier stack a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Each modifier is a pure evaluation node where practical. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- stack owns parameters, not baked output. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- dependencies are explicit. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- modifier can expose edit cage mapping.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Modifier	Own or mediate the Modifier concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ModifierStack	Own or mediate the ModifierStack concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ModifierContext	Own or mediate the ModifierContext concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
GeometryEvalResult	Own or mediate the GeometryEvalResult concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CageMap	Own or mediate the CageMap concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Disabled modifier is identity. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- parameter edit invalidates downstream only. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- evaluator detects dependency cycles. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- modifier cannot mutate source data block.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

15. Modifier Stack - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Boolean fails. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- stack cycle through object reference. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- topology mapping lost. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- high-cost modifier blocks UI.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Stack order tests	Automate the Stack order tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
cache reuse	Automate the cache reuse case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
cycle detection	Automate the cycle detection case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
cancellation	Automate the cancellation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
deterministic evaluation.	Automate the deterministic evaluation. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Modifier API. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- built-ins. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- cache protocol. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- diagnostics. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- stack UI model.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Modifier Stack is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

16. Dependency Graph and Evaluation - Design Specification

Create a scheduler for derived geometry, transforms, constraints, animation, materials, and render extraction.

Source basis: VANTA frame graph demonstrates explicit dependency compilation; concept can inform but not be copied blindly into authoring graph.

Design intent

This subsystem exists to make dependency graph and evaluation a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Graph separates logical dependencies from execution. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- dirty propagation is revision-based. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- evaluation is demand-driven. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- jobs are cancellable. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

- UI never waits on avoidable full-scene recompute.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
DependencyNode	Own or mediate the DependencyNode concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DependencyEdge	Own or mediate the DependencyEdge concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
EvalKey	Own or mediate the EvalKey concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DirtySet	Own or mediate the DirtySet concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
EvalScheduler	Own or mediate the EvalScheduler concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Graph cycles are diagnosed with path. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- evaluation result tagged with inputs/revisions. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- stale async result is discarded.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

16. Dependency Graph and Evaluation - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Hidden cycle. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- race publishes stale mesh. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- excessive invalidation. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- plugin node blocks cancellation.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Cycle fixtures	Automate the Cycle fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
stale-result race	Automate the stale-result race case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
incremental edit benchmark	Automate the incremental edit benchmark case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
graph dump comparison.	Automate the graph dump comparison. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Dependency graph. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- scheduler. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- dirty propagation. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- profiler hooks.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Dependency Graph and Evaluation is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

17. Subdivision Surfaces - Design Specification

Support Catmull-Clark and related subdivision workflows with crease and boundary rules.

Source basis: New geometry subsystem using EditableMesh and compiled MeshAsset.

Design intent

This subsystem exists to make subdivision surfaces a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Separate control cage from evaluated surface. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- adaptive preview levels. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- crease weights and boundary interpolation persisted. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- topology remap available for selection display.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
SubdivisionModifier	Own or mediate the SubdivisionModifier concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CreaseLayer	Own or mediate the CreaseLayer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SubdivEvaluator	Own or mediate the SubdivEvaluator concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
LimitSurfaceSample	Own or mediate the LimitSurfaceSample concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Level zero equals source. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- deterministic vertex ordering. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- crease semantics stable across save/load. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- generated normals consistent with surface.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

17. Subdivision Surfaces - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Extraordinary vertex artifact. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- non-manifold edge unsupported path. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- memory blowup at high level.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Known subdivision fixtures	Automate the Known subdivision fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
crease cases	Automate the crease cases case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
boundary cases	Automate the boundary cases case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
performance at 1M evaluated faces.	Automate the performance at 1M evaluated faces. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Catmull-Clark evaluator. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- adaptive cache. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- crease UI data. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- tests.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Subdivision Surfaces is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

18. Curves and Splines - Design Specification

Provide Bezier, poly, and interpolation curves for modeling, paths, animation, and CAD-adjacent workflows.

Source basis: New authoring type; render path can consume tessellated MeshAsset output.

Design intent

This subsystem exists to make curves and splines a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Curve data stays analytic. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- tessellation is derived. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- handles have aligned/free/vector modes. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- per-spline cyclic state. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- bevel/extrude generate surface output non-destructively.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
CurveData	Own or mediate the CurveData concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Spline	Own or mediate the Spline concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ControlPoint	Own or mediate the ControlPoint concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CurveTessellator	Own or mediate the CurveTessellator concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CurveModifier	Own or mediate the CurveModifier concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Parameter order is stable. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- tessellation tolerance explicit. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- closed curve continuity enforced according to handle mode.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

18. Curves and Splines - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Zero-length segment. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- cusp tangent. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

- self-intersection. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- excessive tessellation from tiny tolerance.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Analytic sample checks	Automate the Analytic sample checks case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
adaptive tessellation	Automate the adaptive tessellation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
save/load handle modes	Automate the save/load handle modes case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
curve-to-mesh.	Automate the curve-to-mesh. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Curve kernel. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- editor handles. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- tessellation. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- conversion commands.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Curves and Splines is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

19. NURBS Curves and Surfaces - Design Specification

Add rational B-spline geometry for precision surface workflows without conflating it with polygon meshes.

Source basis: New geometry code; potential third-party kernel evaluation is part of dependency review.

Design intent

This subsystem exists to make nurbs curves and surfaces a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics.

Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Homogeneous control points. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- explicit degree/knot vectors. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- trim curves separate from base surface. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- tessellation never becomes authoritative unless user converts.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
NurbsCurve	Own or mediate the NurbsCurve concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
NurbsSurface	Own or mediate the NurbsSurface concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
KnotVector	Own or mediate the KnotVector concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
TrimLoop	Own or mediate the TrimLoop concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
NurbsTessellator	Own or mediate the NurbsTessellator concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Knot vector valid and nondecreasing. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- weight finite/nonzero policy. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- surface domain explicit. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- trim orientation validated.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

19. NURBS Curves and Surfaces - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Invalid knot multiplicity. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- singular weights. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- trim self-intersection. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- tessellation cracks at patch boundaries.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
NURBS textbook fixtures	Automate the NURBS textbook fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
rational circle	Automate the rational circle case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
continuity tests	Automate the continuity tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
trim cases	Automate the trim cases case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
patch seam tests.	Automate the patch seam tests. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- NURBS evaluator. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- editor. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- tessellation. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- conversion tools.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

NURBS Curves and Surfaces is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

20. Sculpting - Design Specification

Define multiresolution and dynamic sculpt workflows without destabilizing the base mesh architecture.

Source basis: New subsystem; VANTA rendering and viewport can later display compiled sculpt output.

Design intent

This subsystem exists to make sculpting a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Start with brush deformation on dense mesh. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- multires subdivision before dynamic topology. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- stroke sampling is deterministic enough for undo. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- brush engine supports masks and symmetry.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
SculptSession	Own or mediate the SculptSession concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Brush	Own or mediate the Brush concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
StrokeSample	Own or mediate the StrokeSample concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SculptLayer	Own or mediate the SculptLayer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SymmetryPlane	Own or mediate the SymmetryPlane concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Stroke can be undone. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- mask range bounded. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- symmetry transform stable. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- viewport updates incrementally.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

20. Sculpting - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Huge undo memory. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- brush skips due to event rate. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- symmetry duplicates centerline. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- topology mutation invalidates UVs unexpectedly.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Recorded stroke replay	Automate the Recorded stroke replay case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
pressure curves	Automate the pressure curves case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
symmetry	Automate the symmetry case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
million-vertex latency	Automate the million-vertex latency case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
undo memory.	Automate the undo memory. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Brush core. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- sculpt mode. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- multires integration. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

- performance counters.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Sculpting is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

21. Retopology and Surface Snapping - Design Specification

Support creation of clean production topology over reference surfaces.

Source basis: New editor tools using scene ray queries and EditableMesh.

Design intent

This subsystem exists to make retopology and surface snapping a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Retopo objects remain independent meshes. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- snap target may be any evaluated surface. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- projection modes include nearest/raycast/shrinkwrap. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- symmetry supported.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
SurfaceQuery	Own or mediate the SurfaceQuery concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SnapTarget	Own or mediate the SnapTarget concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
RetopoTool	Own or mediate the RetopoTool concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ShrinkwrapModifier	Own or mediate the ShrinkwrapModifier concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Snapping never modifies target. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- hit uses evaluated target revision. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

- stale BVH result rejected after target changes.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

21. Retopology and Surface Snapping - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Snapping to hidden old mesh. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- ray misses due to precision. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- self-snap loop. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- surface changes during stroke.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Moving target race	Automate the Moving target race case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
backface policy	Automate the backface policy case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
high-density target	Automate the high-density target case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
symmetry retopo.	Automate the symmetry retopo. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- BVH service. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- snap operators. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

- overlay. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- retopo workspace.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Retopology and Surface Snapping is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

22. UV Editing and Mapping - Design Specification

Provide UV layers, islands, seams, unwrap, pack, transforms, and texture-space selection.

Source basis: VANTA MeshAsset currently carries UV0; Cartographer needs arbitrary authoring UV layers and compiler selection.

Design intent

This subsystem exists to make uv editing and mapping a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- UVs are face-corner attributes. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- multiple layers supported. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- seams are edge attributes. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- unwrap runs as command. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- render compiler selects named/default layer.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
UVLayer	Own or mediate the UVLayer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
UVVertex	Own or mediate the UVVertex concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SeamLayer	Own or mediate the SeamLayer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
UVIsland	Own or mediate the UVIsland concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
UnwrapOperator	Own or mediate the UnwrapOperator concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
PackOperator	Own or mediate the PackOperator concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Each loop has UV coordinate per active layer. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- NaNs rejected. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- island graph updates after seam/topology change.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

22. UV Editing and Mapping - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- UV layer cardinality drift. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- pack overlap. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- unwrap singular chart. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- topology op drops UVs.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Cube/cylinder fixtures	Automate the Cube/cylinder fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
seam changes	Automate the seam changes case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
topology propagation	Automate the topology propagation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
pack determinism	Automate the pack determinism case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
UV0 compile.	Automate the UV0 compile. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.

- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- UV data model. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- unwrap/pack. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- editor. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- compiler mapping.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

UV Editing and Mapping is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

23. Materials and Shading Model - Design Specification

Define authoring materials independently from renderer cache and support both simple standard materials and node graphs.

Source basis: VANTA has Material plus sampled base color, normal map, roughness/metallic metadata, alpha modes, and forward rendering foundation.

Design intent

This subsystem exists to make materials and shading model a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Cartographer StandardMaterial uses PBR parameters. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- material data block is shareable. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- renderer compiles material to backend resources. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- material slots are per-face/submesh.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Material	Own or mediate the Material concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
StandardMaterial	Own or mediate the StandardMaterial concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
MaterialSlot	Own or mediate the MaterialSlot concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ShaderCompileKey	Own or mediate the ShaderCompileKey concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
MaterialInstance	Own or mediate the MaterialInstance concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Authoring parameters remain backend neutral. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- missing texture has deterministic fallback. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- material instance override does not mutate parent.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

23. Materials and Shading Model - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Unsupported node silently ignored. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- texture color space wrong. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- slot count mismatch. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- shader compile failure hides geometry.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Reference PBR values	Automate the Reference PBR values case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
missing assets	Automate the missing assets case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
slot remap	Automate the slot remap case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
instance override	Automate the instance override case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
shader fallback.	Automate the shader fallback. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Material core. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- standard shader contract. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- slot editor. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- compiler.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Materials and Shading Model is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

24. Node Material Graph - Design Specification

Provide Blender/Maya-like node-based shading while maintaining a constrained, inspectable compilation model.

Source basis: New public subsystem; renderer backend receives compiled shader/material graph.

Design intent

This subsystem exists to make node material graph a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Typed sockets. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- explicit implicit-conversion rules. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- DAG required for material evaluation. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- graph versioned. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- nodes supplied by core or plugins.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
NodeGraph	Own or mediate the NodeGraph concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Node	Own or mediate the Node concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Socket	Own or mediate the Socket concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Link	Own or mediate the Link concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
TypeSystem	Own or mediate the TypeSystem concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ShaderIR	Own or mediate the ShaderIR concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- No cycles unless a node family explicitly supports feedback in non-material graphs. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- link types validate. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- orphan sockets survive migration safely.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

24. Node Material Graph - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Cycle. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

- missing plugin node. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- shader explosion. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- graph migration loses links. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- invalid implicit conversion.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Graph fuzz	Automate the Graph fuzz case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
missing node placeholder	Automate the missing node placeholder case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
deterministic IR	Automate the deterministic IR case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
shader cache	Automate the shader cache case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
migration fixtures.	Automate the migration fixtures. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Node graph library. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- standard nodes. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- IR. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- material editor.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Node Material Graph is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

25. Texture and Image Pipeline - Design Specification

Handle image assets, color spaces, mip generation, UDIM-like sets, generated images, and reload behavior.

Source basis: VANTA has TGA import, sRGB/linear distinction, mip generation, staged upload, and asset references.

Design intent

This subsystem exists to make texture and image pipeline a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Authoring image supports multiple source formats through adapters. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- original bytes/reference retained separately from GPU texture. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- color-space metadata is explicit. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- reload is digest-based.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
ImageAsset	Own or mediate the ImageAsset concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ImageSource	Own or mediate the ImageSource concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ColorSpace	Own or mediate the ColorSpace concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
TextureSet	Own or mediate the TextureSet concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ImageCache	Own or mediate the ImageCache concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ReloadReceipt	Own or mediate the ReloadReceipt concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Image decode failure does not destroy prior valid version. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- color space is never inferred from filename alone after import. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- path containment enforced.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

25. Texture and Image Pipeline - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Corrupt file. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- huge image allocation. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- changed external asset. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- missing tile. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- path traversal.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Malformed image corpus	Automate the Malformed image corpus case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
reload diff	Automate the reload diff case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
color space fixtures	Automate the color space fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
GPU upload lifecycle	Automate the GPU upload lifecycle case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
missing tile.	Automate the missing tile. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Image core. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- importer adapters. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- cache. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- file watcher later.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Texture and Image Pipeline is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

26. Renderer and RHI - Design Specification

Provide a responsive modeling viewport renderer with clear separation between authoring model and backend resources.

Source basis: VANTA has backend-neutral RHI, frame graph, Vulkan device, renderer, RenderScene, resources, and offscreen targets.

Design intent

This subsystem exists to make renderer and rhi a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Extract/reimplement neutral rendering interfaces. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- editor rendering prioritizes latency and diagnostics. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- render scene is rebuilt from evaluated authoring state. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- Vulkan first, other backends later.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
RenderDevice	Own or mediate the RenderDevice concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
RenderScene	Own or mediate the RenderScene concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DrawPacket	Own or mediate the DrawPacket concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
MeshGpuResource	Own or mediate the MeshGpuResource concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
MaterialGpuResource	Own or mediate the MaterialGpuResource concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
FrameGraph	Own or mediate the FrameGraph concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Renderer never owns authoring truth. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- stale generational handles reject. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- resource destruction waits for GPU completion. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

- validation layer path available.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

26. Renderer and RHI - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Device loss. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- stale mesh resource. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- shader compile error. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- out-of-memory. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- resize race.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Headless render tests	Automate the Headless render tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
resize soak	Automate the resize soak case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
resource lifetime stress	Automate the resource lifetime stress case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
golden images	Automate the golden images case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
device-loss simulation where possible.	Automate the device-loss simulation where possible. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Render library. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- Vulkan backend. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- validation tooling. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- profiling counters.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Renderer and RHI is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

27. Viewport and Editor Overlays - Design Specification

Implement model-space viewport interaction, grids, gizmos, outlines, selection highlighting, guides, and overlays.

Source basis: VANTA has canonical offscreen Viewport and Studio renders through it before presentation.

Design intent

This subsystem exists to make viewport and editor overlays a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Viewport owns render target, not model. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- overlays are separate passes. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- picking uses ID buffer/ray tests. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- high-DPI coordinate conversions centralized.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Viewport	Own or mediate the Viewport concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ViewportCamera	Own or mediate the ViewportCamera concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
OverlayRenderer	Own or mediate the OverlayRenderer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
PickingBuffer	Own or mediate the PickingBuffer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
GizmoLayer	Own or mediate the GizmoLayer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Resize generation invalidates dependent resources. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- overlay never changes model. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- pick result tagged with scene revision.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

27. Viewport and Editor Overlays - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Stale pick after edit. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- DPI offset. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- gizmo occluded incorrectly. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- overlay depth policy inconsistent.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
DPI scales	Automate the DPI scales case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
resize churn	Automate the resize churn case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
selection ID accuracy	Automate the selection ID accuracy case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
orthographic/perspective pick	Automate the orthographic/perspective pick case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
multi-viewport.	Automate the multi-viewport. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Viewport module. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- overlay passes. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- picking. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- gizmo rendering.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Viewport and Editor Overlays is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

28. Lighting, Cameras, and Display Modes - Design Specification

Support production scene cameras plus editor cameras and diagnostic display modes.

Source basis: VANTA has camera projection components, directional lights, forward lighting, normal/depth debug modes.

Design intent

This subsystem exists to make lighting, cameras, and display modes a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Editor camera is not automatically a scene camera. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- scene cameras are objects. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- display modes include solid, material preview, rendered, wireframe, normals, UV stretch, x-ray.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
CameraData	Own or mediate the CameraData concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
LightData	Own or mediate the LightData concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
EditorCamera	Own or mediate the EditorCamera concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DisplayMode	Own or mediate the DisplayMode concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ViewTransform	Own or mediate the ViewTransform concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Near/far valid. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- camera transform double precision in authoring. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- display mode cannot alter material data. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- editor navigation does not dirty project.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

28. Lighting, Cameras, and Display Modes - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Clipping unusable at large scale. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- light count over backend limit. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- editor camera accidentally serialized as scene object.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Projection fixtures	Automate the Projection fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
camera navigation	Automate the camera navigation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
display mode snapshots	Automate the display mode snapshots case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
large scene clipping	Automate the large scene clipping case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
light overflow diagnostics.	Automate the light overflow diagnostics. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Camera/light data. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- editor navigation. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- display modes. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- overlays.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Lighting, Cameras, and Display Modes is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

29. Asset Database and Libraries - Design Specification

Manage reusable meshes, materials, images, rigs, presets, and linked external data.

Source basis: VANTA AssetDatabase and digest-based asset references provide a starting pattern.

Design intent

This subsystem exists to make asset database and libraries a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as

a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Asset identity uses content/reference metadata. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- project references are portable. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- user library is separate from project. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- linked assets are read-only until made local.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
AssetDatabase	Own or mediate the AssetDatabase concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
AssetId	Own or mediate the AssetId concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
AssetReference	Own or mediate the AssetReference concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
LibraryIndex	Own or mediate the LibraryIndex concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
LinkState	Own or mediate the LinkState concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Missing asset is explicit. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- duplicate digest can deduplicate cache without merging logical identity. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- linked asset edits require localization.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

29. Asset Database and Libraries - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Broken path. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

- library version mismatch. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- hash collision assumption. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- asset replaced while open.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Relink	Automate the Relink case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
missing asset	Automate the missing asset case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
duplicate import	Automate the duplicate import case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
library migration	Automate the library migration case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
portability zip.	Automate the portability zip. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Asset DB. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- browser model. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- relink flow. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- library package format.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Asset Database and Libraries is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

30. Import Pipeline - Design Specification

Create a staged, fail-closed importer framework for geometry, scenes, materials, and CAD sources.

Source basis: VANTA glTF importer is bounded and converts importer structures into engine-owned MeshAsset.

Design intent

This subsystem exists to make import pipeline a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Importer parses into temporary foreign model, validates, converts to Cartographer-native data, then commits one transaction. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- import report lists warnings and unsupported data.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Importer	Own or mediate the Importer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ImportContext	Own or mediate the ImportContext concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ForeignScene	Own or mediate the ForeignScene concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ImportReport	Own or mediate the ImportReport concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ImportTransaction	Own or mediate the ImportTransaction concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Failed import leaves document unchanged. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- untrusted file reads are bounded. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- relative external references stay within allowed roots unless user approves.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

30. Import Pipeline - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Malformed offsets. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- decompression bomb. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- path traversal. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- partial scene committed. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- unit mismatch.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Fuzz parsers	Automate the Fuzz parsers case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
corpus import	Automate the corpus import case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
cancellation	Automate the cancellation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
unit tests	Automate the unit tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
security limits	Automate the security limits case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
round-trip where applicable.	Automate the round-trip where applicable. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Importer SDK. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- glTF first. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- OBJ/STL/PLY. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- later USD/STEP/IFC adapters.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Import Pipeline is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

31. Export Pipeline - Design Specification

Produce deterministic, validated outputs for DCC, web, manufacturing, and documentation workflows.

Source basis: VANTA can serialize internal mesh/project data but Cartographer requires broad external export.

Design intent

This subsystem exists to make export pipeline a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Export is explicit projection from authoring state. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- destructive triangulation occurs on temporary evaluated copy. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- report lists dropped/approximated features.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Exporter	Own or mediate the Exporter concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ExportContext	Own or mediate the ExportContext concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ExportPreset	Own or mediate the ExportPreset concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ExportReport	Own or mediate the ExportReport concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SceneProjection	Own or mediate the SceneProjection concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Export never mutates open document. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- units/axes declared. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- unsupported feature never silently disappears without report.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

31. Export Pipeline - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Wrong axis. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- scale mismatch. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- material dropped. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- selection-only export references missing dependency.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Golden exports	Automate the Golden exports case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
reimport comparison	Automate the reimport comparison case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
unit/axis matrix	Automate the unit/axis matrix case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
feature-loss report	Automate the feature-loss report case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
deterministic bytes when format allows.	Automate the deterministic bytes when format allows. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Exporter framework. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- presets. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- validation. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- report UI/CLI.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Export Pipeline is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved,

and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

32. CAD Sketching - Design Specification

Add 2D sketch entities, construction geometry, profiles, dimensions, and support planes.

Source basis: New CAD subsystem layered alongside mesh authoring.

Design intent

This subsystem exists to make cad sketching a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Sketch owns analytic 2D entities in a plane. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- construction entities excluded from profiles. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- constraints separate from geometry. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- solver state is derived.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Sketch	Own or mediate the Sketch concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SketchPlane	Own or mediate the SketchPlane concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Line2	Own or mediate the Line2 concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Arc2	Own or mediate the Arc2 concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Circle2	Own or mediate the Circle2 concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Spline2	Own or mediate the Spline2 concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ConstructionFlag	Own or mediate the ConstructionFlag concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Geometry finite. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- entity IDs stable. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- support plane valid. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

- profile extraction only uses eligible closed loops.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

32. CAD Sketching - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Coincident entities collapse unexpectedly. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- invalid plane. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- profile ambiguity. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- excessive tiny geometry.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Basic sketch fixtures	Automate the Basic sketch fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
plane transforms	Automate the plane transforms case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
profile extraction	Automate the profile extraction case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
save/load	Automate the save/load case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
undo.	Automate the undo. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Sketch kernel. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

- editor. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- profile finder. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- dimensions.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

CAD Sketching is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

33. Geometric Constraint Solver - Design Specification

Solve coincidence, horizontal/vertical, parallel, perpendicular, tangent, equal, radius, distance, angle, symmetry, and fixed constraints.

Source basis: New mathematical subsystem; third-party solver adoption requires license/performance review.

Design intent

This subsystem exists to make geometric constraint solver a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Constraints build residual system. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- solver reports under/fully/over-constrained state. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- user edits become driving goals. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- conflicts identify contributing constraints.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Constraint	Own or mediate the Constraint concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Variable	Own or mediate the Variable concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Residual	Own or mediate the Residual concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SolverResult	Own or mediate the SolverResult concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ConflictSet	Own or mediate the ConflictSet concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DOFReport	Own or mediate the DOFReport concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Solver convergence tolerance explicit. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- failed solve does not commit geometry. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- constraints reference valid sketch entities/parameters.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

33. Geometric Constraint Solver - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Divergence. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- oscillation. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- overconstraint not diagnosed. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- unit scaling destabilizes numeric solve.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Canonical constraint problems	Automate the Canonical constraint problems case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
conflict sets	Automate the conflict sets case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
randomized solvable systems	Automate the randomized solvable systems case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
scale sensitivity	Automate the scale sensitivity case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
performance.	Automate the performance. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.

- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Solver API. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- diagnostics. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- constraint UI. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- benchmark suite.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Geometric Constraint Solver is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

34. B-Rep Geometry Kernel - Design Specification

Represent exact-ish solids and surfaces for CAD operations independently from polygon display meshes.

Source basis: New subsystem; may initially wrap a permissively licensed geometry kernel behind Cartographer-owned interfaces.

Design intent

This subsystem exists to make b-rep geometry kernel a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- BRep uses vertices/edges/wires/faces/shells/solids. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- analytic and NURBS surfaces. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- tessellation is cache. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- topology and geometry tolerance policy centralized.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
BRepShape	Own or mediate the BRepShape concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
BRepVertex	Own or mediate the BRepVertex concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
BRepEdge	Own or mediate the BRepEdge concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
Wire	Own or mediate the Wire concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Face	Own or mediate the Face concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Shell	Own or mediate the Shell concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Solid	Own or mediate the Solid concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Surface	Own or mediate the Surface concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Topology references valid geometry. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- shell closure defined. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- tolerance values bounded. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- tessellation derived and revision-tagged.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

34. B-Rep Geometry Kernel - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Sliver face. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- tolerance explosion. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- invalid wire orientation. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- nonmanifold solid. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- kernel exception.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Primitive solids	Automate the Primitive solids case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
Euler characteristic checks	Automate the Euler characteristic checks case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
corrupted topology	Automate the corrupted topology case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
tessellation consistency	Automate the tessellation consistency case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
import STEP fixtures.	Automate the import STEP fixtures. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Kernel abstraction. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- validators. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- tessellator. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- debug shape dump.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

B-Rep Geometry Kernel is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

35. Solid Modeling and Booleans - Design Specification

Implement extrude, revolve, sweep, loft, fillet/chamfer, shell, split, and boolean operations for precision design.

Source basis: New CAD operators using sketch/NURBS/BRep kernel.

Design intent

This subsystem exists to make solid modeling and booleans a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and

covered by undo, validation, or transaction rules.

Required design decisions

- Features are parametric nodes. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- operation input references persistent subshape naming abstraction. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- failures retain previous valid result and diagnostics.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Feature	Own or mediate the Feature concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ExtrudeFeature	Own or mediate the ExtrudeFeature concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
RevolveFeature	Own or mediate the RevolveFeature concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
BooleanFeature	Own or mediate the BooleanFeature concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
FilletFeature	Own or mediate the FilletFeature concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ShapeReference	Own or mediate the ShapeReference concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Feature result corresponds to input revision. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- failed recompute does not overwrite last valid display unless user chooses. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- units preserved.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

35. Solid Modeling and Booleans - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Topological naming breaks. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

- boolean coincident faces. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- fillet radius too large. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- sweep self-intersection.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Primitive combinations	Automate the Primitive combinations case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
near-coincident booleans	Automate the near-coincident booleans case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
feature parameter sweeps	Automate the feature parameter sweeps case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
naming regression fixtures.	Automate the naming regression fixtures. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Core features. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- diagnostics. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- persistent reference strategy. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- feature UI.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Solid Modeling and Booleans is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

36. Parametric History and Feature Tree - Design Specification

Provide ordered feature history for CAD objects while coexisting with modifier-based DCC workflows.

Source basis: New authoring concept; shares dependency graph and command bus.

Design intent

This subsystem exists to make parametric history and feature tree a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- CAD Body owns feature sequence. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- each feature consumes prior result and named external references. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- suppression/reorder supported with validation. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- conversion to mesh is explicit.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Body	Own or mediate the Body concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
FeatureTree	Own or mediate the FeatureTree concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
FeatureId	Own or mediate the FeatureId concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ExternalReference	Own or mediate the ExternalReference concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SuppressionState	Own or mediate the SuppressionState concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Feature tree acyclic. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- reorder validates dependencies. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- suppressed feature semantics deterministic. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- history edit invalidates downstream only.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

36. Parametric History and Feature Tree - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- External ref deleted. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- reorder creates cycle. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- topological naming remaps wrong face. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- feature plugin missing.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Reorder	Automate the Reorder case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
suppression	Automate the suppression case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
missing ref	Automate the missing ref case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
migration	Automate the migration case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
incremental recompute benchmark.	Automate the incremental recompute benchmark. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Feature tree. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- history panel model. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- dependency integration. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- recovery placeholder nodes.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Parametric History and Feature Tree is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

37. Assemblies and Instancing - Design Specification

Model multi-part assemblies, constraints/mates, references, lightweight representations, and instances.

Source basis: VANTA parent hierarchy and object instances inform generic identity but assembly semantics are new.

Design intent

This subsystem exists to make assemblies and instancing a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Assembly occurrence references part document/data. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- transform/mate solution is derived. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- repeated components share geometry. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- override properties are occurrence-local.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Assembly	Own or mediate the Assembly concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Occurrence	Own or mediate the Occurrence concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
PartReference	Own or mediate the PartReference concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Mate	Own or mediate the Mate concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
MateSolver	Own or mediate the MateSolver concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
LightweightProxy	Own or mediate the LightweightProxy concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Part identity distinct from occurrence identity. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

- mate graph reports conflicts. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- unloaded component retains bounds/metadata.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

37. Assemblies and Instancing - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Circular external reference. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- missing part. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- mate overconstraint. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- part update invalidates huge assembly synchronously.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
1000-instance benchmark	Automate the 1000-instance benchmark case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
missing part	Automate the missing part case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
mate solve	Automate the mate solve case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
part version update	Automate the part version update case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
lightweight load.	Automate the lightweight load. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Assembly core. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- occurrence tree. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- mate subset. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- proxy cache.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Assemblies and Instancing is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

38. Architecture and BIM Objects - Design Specification

Add semantic walls, slabs, roofs, doors, windows, stairs, rooms, levels, grids, and schedules without turning Cartographer into a Revit clone.

Source basis: New domain layer using parametric/BRep/mesh outputs.

Design intent

This subsystem exists to make architecture and bim objects a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Architecture objects preserve semantic parameters. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- generated geometry is derived. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- openings reference host. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- room/space extraction is separate analysis. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- IFC mapping is adapter layer.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Building	Own or mediate the Building concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Level	Own or mediate the Level concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Wall	Own or mediate the Wall concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
Opening	Own or mediate the Opening concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Door	Own or mediate the Door concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Window	Own or mediate the Window concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Slab	Own or mediate the Slab concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Roof	Own or mediate the Roof concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Space	Own or mediate the Space concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Host/opening relations valid. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- level references stable. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- geometric regeneration deterministic. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- semantic object can export without relying on viewport mesh.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

38. Architecture and BIM Objects - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Opening outside host. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- wall join ambiguity. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- self-intersecting footprint. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- regeneration changes object IDs.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Small house fixture	Automate the Small house fixture case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
joins/openings	Automate the joins/openings case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
level changes	Automate the level changes case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
area/volume	Automate the area/volume case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
IFC mapping tests.	Automate the IFC mapping tests. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Architecture module. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- object tools. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- schedules later. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- IFC adapter contract.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Architecture and BIM Objects is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

39. Site, Terrain, and Large Spatial Models - Design Specification

Support terrain meshes/heightfields, parcels, contours, georeferenced origins, and large scenes.

Source basis: VANTA Atlas/region and large-world precision work offers design lessons but should not be exposed wholesale.

Design intent

This subsystem exists to make site, terrain, and large spatial models a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as

a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Authoring uses double-precision coordinates. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- project has optional geospatial reference. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- terrain tiles are data blocks. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- renderer uses local origin rebasing when needed.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
GeoReference	Own or mediate the GeoReference concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Terrain	Own or mediate the Terrain concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
TerrainTile	Own or mediate the TerrainTile concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Contour	Own or mediate the Contour concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SiteBoundary	Own or mediate the SiteBoundary concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
LocalOrigin	Own or mediate the LocalOrigin concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Georeference optional. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- conversion explicit. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- tile seams match. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- render origin changes do not alter authoring coordinates.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

39. Site, Terrain, and Large Spatial Models - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Float jitter. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- mismatched datum assumptions. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- terrain crack. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- huge single mesh allocation.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Kilometer-scale scene	Automate the Kilometer-scale scene case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
tile seam	Automate the tile seam case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
origin shift	Automate the origin shift case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
import DEM	Automate the import DEM case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
save/load georef.	Automate the save/load georef. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Site module. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- terrain data. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- rebasing render adapter. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- georef metadata.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Site, Terrain, and Large Spatial Models is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

40. Measurements, Units, Snapping, and Precision Input - Design Specification

Provide CAD-grade dimensional entry while remaining comfortable for artistic workflows.

Source basis: VANTA uses finite validated vectors but has no units/measurement UI.

Design intent

This subsystem exists to make measurements, units, snapping, and precision input a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Internal geometry uses SI-like canonical units with document display units. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- numeric fields parse unit suffixes. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- snapping services are mode-scoped. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- measurement objects are non-destructive annotations.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
UnitSystem	Own or mediate the UnitSystem concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Quantity	Own or mediate the Quantity concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SnapService	Own or mediate the SnapService concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SnapCandidate	Own or mediate the SnapCandidate concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
MeasureTool	Own or mediate the MeasureTool concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DimensionAnnotation	Own or mediate the DimensionAnnotation concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Unit conversion deterministic. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- snap result includes source/revision. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- numeric parser rejects ambiguous malformed input.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

40. Measurements, Units, Snapping, and Precision Input - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Unit double conversion. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- snap to stale geometry. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- grid rounding accumulates. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- locale decimal parsing conflict.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Imperial/metric fixtures	Automate the Imperial/metric fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
typed expressions	Automate the typed expressions case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
snapping at large coordinates	Automate the snapping at large coordinates case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
tolerance tests.	Automate the tolerance tests. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Unit library. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- numeric parser. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- snap engine. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

- measure tools.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Measurements, Units, Snapping, and Precision Input is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

41. Animation Timeline and Curves - Design Specification

Support keyframes, dope sheet, graph editor, interpolation, constraints, and non-linear clips.

Source basis: New subsystem; scene transforms/material parameters become animatable properties.

Design intent

This subsystem exists to make animation timeline and curves a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Property addressing uses stable IDs/paths. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- F-curves are data blocks. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- evaluation happens before dependency graph consumers. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- editor scrubbing never commits baked values to base property.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
AnimPropertyPath	Own or mediate the AnimPropertyPath concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Keyframe	Own or mediate the Keyframe concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
FCurve	Own or mediate the FCurve concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Action	Own or mediate the Action concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Clip	Own or mediate the Clip concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Timeline	Own or mediate the Timeline concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- At a given time evaluation deterministic. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- missing target yields diagnostic not crash. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- key edits undo. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- interpolation modes persist.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

41. Animation Timeline and Curves - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Object renamed breaks path. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- time precision drift. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- cyclic modifier recursion. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- plugin property unavailable.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Interpolation fixtures	Automate the Interpolation fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
rename stability	Automate the rename stability case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
frame-rate independence	Automate the frame-rate independence case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
clip blending	Automate the clip blending case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
save/load.	Automate the save/load. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.

- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Animation core. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- timeline model. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- graph editor backend. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- playback clock.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Animation Timeline and Curves is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

42. Rigging and Skinning - Design Specification

Provide armatures, bones, constraints, inverse kinematics, skin weights, and deformation stack.

Source basis: New subsystem using scene hierarchy and modifier/dependency graph.

Design intent

This subsystem exists to make rigging and skinning a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Armature data separate from pose state. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- bone identity stable. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- weights sparse and normalized by policy. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- deformation produces derived mesh. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- constraints solved in pose graph.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Armature	Own or mediate the Armature concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Bone	Own or mediate the Bone concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Pose	Own or mediate the Pose concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SkinWeights	Own or mediate the SkinWeights concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ArmatureModifier	Own or mediate the ArmatureModifier concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
IKConstraint	Own or mediate the IKConstraint concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- No bone cycles. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- weights finite/nonnegative. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- bind matrices valid. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- missing bone weight groups handled explicitly.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

42. Rigging and Skinning - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- IK divergence. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- zero-sum weights. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- bone deletion loses weights. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- non-uniform scale artifacts.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Two-bone IK	Automate the Two-bone IK case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
long chains	Automate the long chains case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
weight normalization	Automate the weight normalization case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
bind/rest pose	Automate the bind/rest pose case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
mirrored rig.	Automate the mirrored rig. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Armature core. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- skinning CPU then GPU. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- pose editor. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- basic constraints.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Rigging and Skinning is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

43. Deformers and Blend Shapes - Design Specification

Support lattice, bend/twist, shrinkwrap, simple deformers, shape keys, and morph animation.

Source basis: New modifier/deformation subsystem.

Design intent

This subsystem exists to make deformers and blend shapes a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as

a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Deformer is evaluation node. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- shape key topology must match basis. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- deltas stored compactly. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- destructive topology changes require explicit shape-key policy.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Deformer	Own or mediate the Deformer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ShapeKeySet	Own or mediate the ShapeKeySet concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ShapeKey	Own or mediate the ShapeKey concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DeltaBuffer	Own or mediate the DeltaBuffer concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Lattice	Own or mediate the Lattice concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Shape keys share topology generation. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- weights bounded by configurable policy. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- stale keys flagged after incompatible edit.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

43. Deformers and Blend Shapes - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Topology edit silently corrupts morph. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

- huge sparse delta memory. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- self-referential shrinkwrap.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Shape interpolation	Automate the Shape interpolation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
topology mismatch	Automate the topology mismatch case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
deformer order	Automate the deformer order case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
animated weights	Automate the animated weights case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
sparse storage.	Automate the sparse storage. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Shape keys. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- core deformers. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- topology compatibility diagnostics.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Deformers and Blend Shapes is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

44. Physics and Simulation - Design Specification

Provide optional rigid body and later cloth/soft-body simulation as derived authoring tools, not authoritative project mutation loops.

Source basis: VANTA has RigidBodyConfig, ColliderConfig, PhysicsAdapter seam, deterministic reference physics, and physics system orchestration.

Design intent

This subsystem exists to make physics and simulation a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Simulation runs in session state. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- bake produces explicit cache/action. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- authoring objects define simulation inputs. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- backend adapter permits external physics library later.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
RigidBodySettings	Own or mediate the RigidBodySettings concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Collider	Own or mediate the Collider concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SimulationWorld	Own or mediate the SimulationWorld concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
PhysicsAdapter	Own or mediate the PhysicsAdapter concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
BakeCache	Own or mediate the BakeCache concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Simulation cannot silently rewrite base transforms unless bake/apply command. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- backend metadata recorded. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- non-finite step rejected.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

44. Physics and Simulation - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Backend divergence. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- unstable timestep. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- bake references stale topology. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- collider out of date.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Deterministic reference cases	Automate the Deterministic reference cases case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
fixed-step	Automate the fixed-step case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
backend cross-check	Automate the backend cross-check case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
bake invalidation	Automate the bake invalidation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
collision primitives.	Automate the collision primitives. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Physics adapter. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- reference backend. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- bake cache. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- UI controls.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Physics and Simulation is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved,

and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

45. Project Persistence, Migration, and Atomic Save - Design Specification

Implement robust save/open for very large projects with future chunking and externalized caches.

Source basis: VANTA save writes temp sibling, flushes, reparses, then atomically replaces target; format migrations fail closed.

Design intent

This subsystem exists to make project persistence, migration, and atomic save a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Adopt atomic commit. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- separate manifest from large binary chunks when scale requires. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- checksums on chunks. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- journal/recovery metadata. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- schema migrations transactional.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
ProjectWriter	Own or mediate the ProjectWriter concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ProjectReader	Own or mediate the ProjectReader concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ChunkStore	Own or mediate the ChunkStore concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Migration	Own or mediate the Migration concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SaveReceipt	Own or mediate the SaveReceipt concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Existing valid file survives failed save. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- reader bounds allocations. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- chunk checksum verified. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

- migration either succeeds completely or leaves original untouched.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

45. Project Persistence, Migration, and Atomic Save - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Disk full. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- crash mid-save. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- corrupted chunk. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- partial migration. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- path traversal in external references.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Fault-injection filesystem	Automate the Fault-injection filesystem case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
checksum corruption	Automate the checksum corruption case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
multi-version fixtures	Automate the multi-version fixtures case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
10GB synthetic project metadata.	Automate the 10GB synthetic project metadata. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Persistence library. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- recovery journal. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- migration tests. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- integrity tool.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Project Persistence, Migration, and Atomic Save is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

46. Autosave and Crash Recovery - Design Specification

Protect work without turning autosave into hidden destructive state or uncontrolled disk growth.

Source basis: New editor service built on atomic persistence and command history.

Design intent

This subsystem exists to make autosave and crash recovery a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Autosave uses snapshot/journal in recovery directory. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- never overwrites named project directly. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- cadence based on dirty state/time. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- retention bounded. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- recovery prompts compare revisions.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
AutosaveService	Own or mediate the AutosaveService concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
RecoverySnapshot	Own or mediate the RecoverySnapshot concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
RecoveryIndex	Own or mediate the RecoveryIndex concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
DirtyTracker	Own or mediate the DirtyTracker concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Recovery file identifies source project and revision. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- successful manual save advances recovery baseline. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- stale recovery can be discarded safely.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

46. Autosave and Crash Recovery - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Autosave during huge operation stalls UI. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- recovery newer but incompatible. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- disk full. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- repeated crash loop.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Crash simulation	Automate the Crash simulation case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
stale recovery	Automate the stale recovery case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
retention	Automate the retention case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
concurrent project instances	Automate the concurrent project instances case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
manual save race.	Automate the manual save race. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.

- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Autosave service. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- recovery browser. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- retention policy. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- telemetry-free diagnostics.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Autosave and Crash Recovery is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

47. Plugin System - Design Specification

Allow importers, exporters, modifiers, tools, panels, and data translators without handing plugins raw unsafe ownership of core internals.

Source basis: VANTA SDK plugin descriptor is data-only and deliberately avoids World pointers/backends.

Design intent

This subsystem exists to make plugin system a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- C ABI or stable narrow boundary for loader. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- versioned descriptors. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- capability declarations. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- plugin calls guarded. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- in-process native plugins first, sandbox optional later.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
PluginDescriptor	Own or mediate the PluginDescriptor concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
PluginHost	Own or mediate the PluginHost concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ToolPlugin	Own or mediate the ToolPlugin concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ImporterPlugin	Own or mediate the ImporterPlugin concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ModifierPlugin	Own or mediate the ModifierPlugin concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
PanelPlugin	Own or mediate the PanelPlugin concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- ABI version checked before load. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- plugin cannot register duplicate stable ID. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- unload only when no owned objects/tasks remain.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

47. Plugin System - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Crash/exception crosses boundary. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- plugin unload leaves callbacks. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- incompatible compiler ABI. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- malicious file importer.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Version mismatch	Automate the Version mismatch case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
load/unload cycles	Automate the load/unload cycles case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
exception guard	Automate the exception guard case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
plugin-owned node placeholder after uninstall.	Automate the plugin-owned node placeholder after uninstall. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- SDK headers. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- loader. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- sample plugin. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- compatibility policy.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Plugin System is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

48. Python Scripting API - Design Specification

Expose high-level authoring automation without making Python a runtime dependency of core libraries.

Source basis: New optional embedding layer; user preference permits Python compatibility.

Design intent

This subsystem exists to make python scripting api a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Python module wraps stable command/document APIs. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- scripts default to same undo transaction semantics. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- headless mode can run scripts. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- object handles validate generation.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
python cartographer module	Own or mediate the python cartographer module concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ScriptContext	Own or mediate the ScriptContext concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
TransactionContext	Own or mediate the TransactionContext concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
PyObjectHandle	Own or mediate the PyObjectHandle concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Script cannot bypass validation through raw pointers. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- exception rolls back active transaction. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- versioned API reports capability availability.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

48. Python Scripting API - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Long script blocks UI. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- stale handle. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- extension module ABI mismatch. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- script performs partial changes before error.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Undoable script	Automate the Undoable script case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
exception rollback	Automate the exception rollback case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
headless script	Automate the headless script case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
API introspection	Automate the API introspection case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
stale handle.	Automate the stale handle. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Python bindings. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- console. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- script runner. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- docs/examples.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Python Scripting API is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

49. C++ SDK and Embedding - Design Specification

Offer native consumers access to geometry, scene, IO, and headless conversion without requiring the desktop app.

Source basis: VANTA uses header-only SDK compatibility tokens and split libraries.

Design intent

This subsystem exists to make c++ sdk and embedding a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and

covered by undo, validation, or transaction rules.

Required design decisions

- Install exported targets. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- public ABI policy explicit. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- opaque handles at plugin boundary. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- normal library consumers may use C++ API with semantic versioning.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
carto::sdk::Version	Own or mediate the carto::sdk::Version concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ProjectApi	Own or mediate the ProjectApi concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
GeometryApi	Own or mediate the GeometryApi concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
IoRegistry	Own or mediate the IoRegistry concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
HeadlessContext	Own or mediate the HeadlessContext concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Public headers avoid platform internals. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- API compatibility checks fail clearly. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- feature macros reflect build.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

49. C++ SDK and Embedding - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- ABI break in patch release. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- consumer accidentally depends on internal header. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

- symbol collision.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Install-tree sample	Automate the Install-tree sample case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
ABI checker	Automate the ABI checker case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
header self-containment	Automate the header self-containment case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
minimal consumer builds.	Automate the minimal consumer builds. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- SDK package. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- examples. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- API policy. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- symbol visibility rules.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

C++ SDK and Embedding is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

50. Headless CLI and Batch Processing - Design Specification

Support conversion, validation, rendering previews, mesh analysis, project repair checks, and automation on machines without GUI.

Source basis: VANTA has headless acceptance and dataset validators demonstrating GUI-independent targets.

Design intent

This subsystem exists to make headless cli and batch processing a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and

covered by undo, validation, or transaction rules.

Required design decisions

- CLI commands reuse same libraries. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- operations are deterministic where possible. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- structured JSON report option. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- exit codes stable. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- no GPU required for non-render commands.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
cartographer-cli	Own or mediate the cartographer-cli concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
validate	Own or mediate the validate concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
convert	Own or mediate the convert concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
inspect	Own or mediate the inspect concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
render	Own or mediate the render concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
repair-check	Own or mediate the repair-check concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- CLI never requires user config for deterministic conversion. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- errors identify file and operation. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- batch isolates per-file failures.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

50. Headless CLI and Batch Processing - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- One bad file aborts entire batch. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- GUI-only dependency sneaks in. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- stdout mixes logs and JSON.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Batch corpus	Automate the Batch corpus case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
no-display CI	Automate the no-display CI case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
exit codes	Automate the exit codes case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
JSON schema	Automate the JSON schema case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
cancellation.	Automate the cancellation. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- CLI executable. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- command registry. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- report schema. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- shell completions later.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Headless CLI and Batch Processing is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

51. Performance and Memory Architecture - Design Specification

Set latency and memory budgets for interactive modeling across small and very large scenes.

Source basis: VANTA already tracks resource lifetimes, GPU timestamps, frame receipts, and bounded work in several subsystems.

Design intent

This subsystem exists to make performance and memory architecture a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Measure before optimizing. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- caches revision-keyed. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- geometry uses pooled allocations. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- evaluated meshes may be evicted. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- UI frame target 16.6 ms where feasible, tool feedback under 50 ms.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Profiler	Own or mediate the Profiler concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
MemoryTracker	Own or mediate the MemoryTracker concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CacheBudget	Own or mediate the CacheBudget concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
EvalStats	Own or mediate the EvalStats concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
FrameStats	Own or mediate the FrameStats concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Every major cache reports bytes and revision. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- budget eviction never evicts authoritative data. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- profiler can be disabled with low overhead.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

51. Performance and Memory Architecture - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Cache thrash. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- unbounded undo. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- modifier copies full mesh repeatedly. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- GPU duplicate resources.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
1M/10M polygon scenes	Automate the 1M/10M polygon scenes case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
10k objects	Automate the 10k objects case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
memory leak soak	Automate the memory leak soak case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
modifier stack benchmark	Automate the modifier stack benchmark case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
import large file.	Automate the import large file. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Performance dashboard. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

- benchmark suite. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- budgets. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- regression thresholds.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Performance and Memory Architecture is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

52. Threading and Job System - Design Specification

Keep expensive tessellation, modifier evaluation, imports, thumbnails, and analysis off the UI thread while preserving correctness.

Source basis: VANTA generation runtime demonstrates bounded caller-pumped queues and stale-revision cancellation concepts.

Design intent

This subsystem exists to make threading and job system a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Jobs carry input revisions. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- results publish only if still current. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- priority classes. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- cooperative cancellation. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- core model mutations return to main transaction lane.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
JobSystem	Own or mediate the JobSystem concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
JobHandle	Own or mediate the JobHandle concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CancellationToken	Own or mediate the CancellationToken concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
RevisionFence	Own or mediate the RevisionFence concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
TaskPriority	Own or mediate the TaskPriority concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- No background thread mutates authoritative document directly. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- stale results discarded. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- shutdown joins/cancels safely.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

52. Threading and Job System - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Use-after-close project. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- publish stale result. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- priority inversion. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- deadlock between renderer and geometry task.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Race harness	Automate the Race harness case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
repeated open/close	Automate the repeated open/close case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
cancellation storm	Automate the cancellation storm case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
TSAN-capable build where possible	Automate the TSAN-capable build where possible case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
deterministic single-thread mode.	Automate the deterministic single-thread mode. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.

- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Job library. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- task queues. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- main-thread publish. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- diagnostics.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Threading and Job System is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

53. Large Scene and Streaming Strategy - Design Specification

Scale to large environments, architecture projects, and assemblies without loading every evaluated asset at full fidelity.

Source basis: VANTA Atlas, region packs, large-world precision, and materialization provide lessons for identity/residency boundaries.

Design intent

This subsystem exists to make large scene and streaming strategy a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Cartographer uses generic chunk/residency manager, not VANTA procedural Atlas. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- authoritative object metadata remains available. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- heavy geometry/image data can be unloaded. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- proxies retain bounds.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
ResidencyManager	Own or mediate the ResidencyManager concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
ChunkRef	Own or mediate the ChunkRef concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ProxyGeometry	Own or mediate the ProxyGeometry concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
StreamingBudget	Own or mediate the StreamingBudget concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
LocalOrigin	Own or mediate the LocalOrigin concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Unloaded object retains identity/transform/bounds. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- reload verifies content digest. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- save does not depend on viewport residency state.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

53. Large Scene and Streaming Strategy - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Missing chunk. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- proxy mismatch. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- camera crosses origin. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- thrash around budget threshold.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Million-object metadata synthetic test	Automate the Million-object metadata synthetic test case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
unload/reload	Automate the unload/reload case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
origin rebasing	Automate the origin rebasing case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
missing chunk	Automate the missing chunk case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
memory budget.	Automate the memory budget. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Streaming layer. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- proxy format. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- residency UI. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- large scene benchmark.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Large Scene and Streaming Strategy is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

54. Testing, Fuzzing, and Verification - Design Specification

Define a verification culture suitable for a geometry application where silent corruption is worse than obvious failure.

Source basis: VANTA already contains focused tests for world, hierarchy, mesh assets, render scene, project, studio editor, viewport, physics, and more.

Design intent

This subsystem exists to make testing, fuzzing, and verification a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as

a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Unit tests by narrow target. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- geometry invariants run in debug/tests. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- fuzz untrusted parsers and topology ops. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- golden assets stored compactly. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- visual tests tolerate controlled backend variance.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
TestFixture	Own or mediate the TestFixture concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
FuzzHarness	Own or mediate the FuzzHarness concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
GoldenMesh	Own or mediate the GoldenMesh concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
GoldenImage	Own or mediate the GoldenImage concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
PropertyTest	Own or mediate the PropertyTest concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Every bug that corrupted data gains regression fixture. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- failing invariant identifies object/operator. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- test assets have provenance/license.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

54. Testing, Fuzzing, and Verification - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Golden images too brittle. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- fuzz result unreproducible. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- huge binary fixtures. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- tests require discrete GPU.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Seed recording	Automate the Seed recording case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
CPU/headless paths	Automate the CPU/headless paths case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
topology fuzz	Automate the topology fuzz case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
parser fuzz	Automate the parser fuzz case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
migration matrix	Automate the migration matrix case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
render tolerance masks.	Automate the render tolerance masks. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Test framework conventions. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- corpora. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- CI jobs. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- bug-to-regression policy.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Testing, Fuzzing, and Verification is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

55. CI, Packaging, and Releases - Design Specification

Build trustworthy public releases with reproducible metadata, signed artifacts later, and clear platform support.

Source basis: New public delivery layer using CMake targets and headless tests.

Design intent

This subsystem exists to make ci, packaging, and releases a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- CI builds debug/release. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- Windows primary initially. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- Linux headless/core early. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- dependency lock manifests. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- release bundles include licenses and version report.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
BuildPreset	Own or mediate the BuildPreset concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
PackageManifest	Own or mediate the PackageManifest concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
VersionInfo	Own or mediate the VersionInfo concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ReleaseChecklist	Own or mediate the ReleaseChecklist concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Version embedded in app/CLI. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- package contains exact dependency notices. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- release cannot publish with failing required tests.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor

stable IDs or generation-checked handles over long-lived raw pointers.

55. CI, Packaging, and Releases - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Works only on developer machine. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- DLL missing. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- shader assets absent. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- license file omitted.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Clean VM install	Automate the Clean VM install case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
portable package	Automate the portable package case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
upgrade/downgrade project fixture	Automate the upgrade/downgrade project fixture case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
offline startup	Automate the offline startup case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
checksum manifest.	Automate the checksum manifest. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- CI workflows. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- packaging scripts. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- release checklist. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- support matrix.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

CI, Packaging, and Releases is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

56. Licensing and Community Governance - Design Specification

Choose a permissive open-source posture while protecting project identity and preventing accidental inclusion of incompatible code.

Source basis: New governance; all extracted code requires provenance review.

Design intent

This subsystem exists to make licensing and community governance a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Prefer permissive license such as Apache-2.0 or MIT after counsel/business decision. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- CLA/DCO choice explicit. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- trademarks separate from code license. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- dependency policy documented.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
LICENSE	Own or mediate the LICENSE concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
NOTICE	Own or mediate the NOTICE concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CONTRIBUTING	Own or mediate the CONTRIBUTING concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
SECURITY	Own or mediate the SECURITY concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CODE_OF_CONDUCT	Own or mediate the CODE_OF_CONDUCT concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
DependencyPolicy	Own or mediate the DependencyPolicy concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Every source file/repo has declared license. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

- third-party notices generated. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- contributions attest origin under selected process.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

56. Licensing and Community Governance - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- GPL-only dependency contaminates distribution goals. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- contributor submits copied proprietary code. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- name/trademark ambiguity.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Automated license scan	Automate the Automated license scan case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
dependency allowlist	Automate the dependency allowlist case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
provenance checklist	Automate the provenance checklist case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
release notice validation.	Automate the release notice validation. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Governance files. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

- contributor guide. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- security policy. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- release license report.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Licensing and Community Governance is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

57. VANTA Extraction Plan - Design Specification

Translate the inspected private VANTA modules into a safe reusable foundation without making Cartographer a VANTA fork.

Source basis: Inspected targets include vanta_world, vanta_assets, vanta_render, vanta_project, vanta_studio_session; source paths include world/components/mutation, MeshAsset/glTF, Viewport/RenderScene, ProjectDocument, command bus/inspector.

Design intent

This subsystem exists to make vanta extraction plan a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Extract concepts in dependency order. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- rename namespaces/types. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- remove ML/generation/trace assumptions. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- introduce double-precision authoring layer rather than copying float World transform wholesale. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- keep VANTA adapters private.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
SourceMap	Own or mediate the SourceMap concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
PublicReplacement	Own or mediate the PublicReplacement concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
VantaAdapter	Own or mediate the VantaAdapter concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CompatibilityTest	Own or mediate the CompatibilityTest concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- No public target links private VANTA. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- reused algorithms have provenance. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- public tests assert expected generic behavior without private fixtures.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

57. VANTA Extraction Plan - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Straight copy carries private comments. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- semantic coupling to World revision model blocks DCC needs. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- float transform precision adopted accidentally.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Fresh repo build	Automate the Fresh repo build case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
symbol scan	Automate the symbol scan case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
behavior compatibility tests	Automate the behavior compatibility tests case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
provenance review	Automate the provenance review case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
public-header review.	Automate the public-header review. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Extraction matrix. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- staged commits. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- compatibility harness. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- declassification review checklist.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

VANTA Extraction Plan is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

58. Open World Harness and Forge Integration - Design Specification

Use existing Unreal/Unity integration experience and ForgeOfFate project assets as downstream validation targets rather than dependencies.

Source basis: Open-World-Model-Harness contains substantial Unreal/Unity adapters and ForgeOfFate contains a real Unreal project.

Design intent

This subsystem exists to make open world harness and forge integration a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Cartographer exports/bridges through neutral interchange. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- no gameplay harness code enters geometry core. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- Unreal/Unity examples validate scene/asset portability. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- Forge serves as a representative consumer.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
UnrealExporter	Own or mediate the UnrealExporter concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
UnityExporter	Own or mediate the UnityExporter concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
InterchangeManifest	Own or mediate the InterchangeManifest concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Interface / type	Responsibility
ValidationProject	Own or mediate the ValidationProject concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Engine integration is optional. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- exported data has declared coordinate/unit conventions. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- consumer-specific metadata lives in adapters.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

58. Open World Harness and Forge Integration - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Exporter assumes private gameplay types. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- asset appears correct in Cartographer but broken in Unreal. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- coordinate handedness mismatch.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Export sample scene to Unreal/Unity	Automate the Export sample scene to Unreal/Unity case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
material/mesh parity	Automate the material/mesh parity case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
scale/axis checks	Automate the scale/axis checks case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
animation later.	Automate the animation later. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.

- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Integration samples. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- documentation. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- consumer acceptance fixtures.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Open World Harness and Forge Integration is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

59. Roadmap from 0.1 to 1.0 - Design Specification

Sequence delivery so Cartographer becomes useful early without attempting Blender, Maya, AutoCAD, and Revit simultaneously.

Source basis: Based on inspected VANTA foundation and missing DCC/CAD authoring layers.

Design intent

This subsystem exists to make roadmap from 0.1 to 1.0 a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- 0.1 focuses scene/mesh/edit/render/project. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- 0.2 modifiers/UV/material maturity. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- 0.3 animation/rigging basics. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- 0.4 CAD sketch/constraints. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- 0.5 BRep/features. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- later architecture and cross-platform depth. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- 1.0 requires stable format/SDK.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
Milestone	Own or mediate the Milestone concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Epic	Own or mediate the Epic concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
CapabilityGate	Own or mediate the CapabilityGate concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
ReleaseCriteria	Own or mediate the ReleaseCriteria concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- Each milestone has coherent user workflow. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- deferred features fail visibly. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- project migrations preserve prior work. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- version claims match tests.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

59. Roadmap from 0.1 to 1.0 - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Roadmap driven by breadth not workflow. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- BRep swallows project. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- UI built before geometry invariants. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- unstable file format released as 1.0.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
Milestone acceptance scenes	Automate the Milestone acceptance scenes case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
upgrade path	Automate the upgrade path case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
release checklist	Automate the release checklist case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
capability matrix.	Automate the capability matrix. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Epic map. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- milestone criteria. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- dependency graph. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- 1.0 exit criteria.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Roadmap from 0.1 to 1.0 is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

60. Risk Register and Final Acceptance - Design Specification

Make technical risks explicit and define what counts as a credible first public release.

Source basis: Synthesizes all repo findings and proposed architecture.

Design intent

This subsystem exists to make risk register and final acceptance a first-class, testable part of Cartographer rather than a collection of UI shortcuts. The implementation must preserve the separation between authoritative authoring data, derived evaluation results, renderer resources, and user-interface state. That separation is central to the entire product: a viewport, cache, importer, plugin, or tool may propose or derive data, but it must not become the only place the user's work exists.

The preferred implementation style is explicit and narrow. Public types should state ownership, lifetime, revision, and failure semantics. Expensive or lossy transformations should return reports or receipts that make approximation visible. Where a subsystem can be written as a pure or deterministic transformation, it should be; where mutable state is unavoidable, the mutation boundary must be obvious and covered by undo, validation, or transaction rules.

Required design decisions

- Top risks: geometry kernel correctness, CAD scope, UI complexity, cross-platform rendering, file-format stability, dependency licensing, performance, contributor maintainability. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.
- acceptance emphasizes honest capability labels.. The implementation should record this as an architecture decision if later work changes the rule, because downstream file formats, plugins, or workflows may depend on it.

Primary interfaces and data ownership

The initial public vocabulary for this area should include the following named interfaces or data types. Names may change during implementation, but their responsibilities should remain separated:

Interface / type	Responsibility
RiskItem	Own or mediate the RiskItem concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Severity	Own or mediate the Severity concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Likelihood	Own or mediate the Likelihood concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
Mitigation	Own or mediate the Mitigation concern. It must expose validated operations and avoid leaking unrelated backend or UI state.
AcceptanceGate	Own or mediate the AcceptanceGate concern. It must expose validated operations and avoid leaking unrelated backend or UI state.

Authoritative invariants

- No critical unresolved data-corruption risk at release. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- known limitations documented. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.
- release acceptance tied to tests and sample workflows, not feature count.. Validation should run at construction/import/commit boundaries, and a debug validation path should be available after complex operators.

Implementation notes

Cartographer should prefer revision-tagged derived data. Every expensive output - tessellation, compiled mesh, acceleration structure, shader, thumbnail, evaluated modifier result, or import conversion - should carry enough source identity to determine whether it is still current before publication. This avoids a common desktop-editor failure mode where asynchronous work completes after the user has changed the model and quietly replaces newer state.

Memory ownership must also be visible. Authoritative document objects belong to the document/model layer; caches belong to subsystem services; GPU resources belong to the renderer; panes and tools own only transient interaction state. Cross-layer references should favor stable IDs or generation-checked handles over long-lived raw pointers.

60. Risk Register and Final Acceptance - Verification and Delivery Specification

Failure model

Verification for this subsystem is not limited to happy-path unit tests. The engineering target is that malformed input, stale asynchronous work, unsupported data, resource exhaustion, and user cancellation produce explicit failure without corrupting the open project. The following failure modes are mandatory design cases, not post-release edge cases:

- Silent corruption. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- marketing outruns capability. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- unsupported files accepted partially. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- crash recovery untested. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.
- open-source extraction leaks private code.. The expected response must specify whether the operation is rejected, partially degraded with a report, retried, or replaced by a safe fallback. Silent success is not acceptable.

Test plan

Test family	Minimum acceptance behavior
End-to-end modeling workflows	Automate the End-to-end modeling workflows case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
crash/recovery	Automate the crash/recovery case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
import/export	Automate the import/export case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
stress	Automate the stress case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
license audit	Automate the license audit case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
clean build	Automate the clean build case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.
project migration.	Automate the project migration. case with deterministic fixtures or recorded seeds. Failures must preserve model integrity and emit actionable diagnostics.

Cross-cutting verification

- Undo/redo: when the subsystem changes authoring state, execute, undo, and redo must round-trip the relevant state or explicitly document irreversible operations.
- Persistence: save/open must preserve all authoritative parameters, stable IDs required by downstream references, and feature/version metadata.
- Determinism: operations that claim deterministic output must be compared byte-for-byte or by canonicalized semantic digest across repeated runs.
- Performance: benchmark with both representative and adversarial scale; report hardware and dataset size rather than publishing context-free numbers.
- Diagnostics: every rejection path returns or records a reason suitable for the Problems panel and headless CLI.

Delivery artifacts

- Risk register. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- release gates. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- known limitations. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.
- final acceptance matrix.. This artifact is part of subsystem completion and should be reviewed alongside the code, not created as a release afterthought.

Definition of done

Risk Register and Final Acceptance is considered implemented only when its public contract is documented, the narrow library target builds independently, invalid inputs are rejected safely, required persistence paths are covered, and its acceptance tests run in CI without private repositories. A UI mockup or one successful demonstration is not sufficient.

For public releases, known limitations should be written in capability language: what data is accepted, what is generated, what is preserved, and what is not supported. This is especially important for geometry and interchange features, where users can otherwise mistake partial parsing or approximate export for full semantic compatibility.

Appendix A - Proposed Initial C++ Target Graph

```
carto_core
  |-- carto_scene
  |-- carto_project
  |-- carto_geometry
  |     |-- carto_mesh
  |     |-- carto_curves
  |     |-- carto_modifiers
  |     `-- carto_cad (optional early)
  |-- carto_rhi
  |     `-- carto_gpu_graph
  |           `-- carto_render
  |-- carto_io
  |-- carto_animation
  |-- carto_editor
  `-- cartographer_app

Optional adapters: carto_vulkan, carto_python, carto_unreal_export, carto_ifc, carto_step
```

The graph should be enforced in CMake and by header ownership. In particular, carto_geometry must build without the renderer or application shell, and carto_render must consume compiled geometry without including editable topology internals. This makes headless tests fast and keeps the public libraries useful outside the desktop application.

Appendix B - Suggested Project Directory Layout

```
/apps/cartographer
/apps/cli
/libs/core
/libs/scene
/libs/project
/libs/geometry/mesh
/libs/geometry/curves
/libs/geometry/modifiers
/libs/cad/sketch
/libs/cad/brep
/libs/animation
/libs/rhi
/libs/gpu_graph
/libs/render
/libs/io
/libs/editor
/plugins/importers
/plugins/exporters
/samples
/tests/fixtures
/benchmarks
/docs/architecture
/docs/adr
```

Keep generated build artifacts, downloaded SDKs, and large binary acceptance data outside the source tree. Small permissively licensed fixtures may live under tests/fixtures with a manifest containing source, license, and expected behavior.

Appendix C - Definition of the First Useful User Workflow

- Launch without login or network access and create a new project.
- Add a primitive mesh, enter component mode, select faces, extrude/inset/bevel, and adjust transforms numerically.
- Add a mirror and subdivision modifier, toggle them non-destructively, and undo/redo both parameter and topology edits.
- Assign a standard material and texture, inspect the result in a Vulkan viewport, and switch wireframe/solid/material display modes.
- Save atomically, close, reopen, and reproduce the same scene/object/modifier/material state.
- Export glTF or OBJ, reimport the export into a new project, and receive a clear feature-loss report where round-trip semantics differ.
- Recover from an injected failed save without losing the prior valid project file.

If release 0.1 can complete this workflow reliably, Cartographer is already a real open-source modeling application. CAD/BIM and animation can then deepen the product without destabilizing the fundamental document and geometry architecture.

Appendix D - Final Engineering Principles

Principle	Requirement
Standalone first	Cartographer builds and runs independently of HAVEN, VANTA, RegOS, IDA, or any defense application.
Authoring truth is explicit	Meshes, BReps, curves, sketches, animation, and semantic objects have authoritative models separate from render caches.
Derived data is disposable	A project can regenerate view meshes, BVHs, thumbnails, shader caches, and viewport resources.
Failure is visible	Unsupported data, stale async work, geometry failure, and import loss are reported instead of silently approximated.
Undo is architecture	Undo/redo is not a UI afterthought; mutation APIs are designed around reversible transactions.
Precision is intentional	Double-precision authoring and explicit units protect CAD, architecture, and large-scene workflows.
Open source is real	A public clone can compile, test, model, save, and export without access to private Hidden Canopy repositories.
Scope is staged	The project earns breadth through stable foundations rather than shipping four half-built applications in one shell.

End of master engineering specification. Implementation should convert major irreversible decisions into ADRs and keep this document versioned alongside the public roadmap.